

(and likely execution cycles), but it will still be much smaller than a program with 32-bit Arm instructions.

Operating Behaviors

A CPU may have several operating modes with different capabilities to provide better performance, more safety, or additional features.

Thread and Handler Modes

The processor normally runs in Thread mode, but enters Handler mode when servicing exceptions and interrupts. The Thread mode simplifies the creation of multitasking systems. Thread and Handler modes differ in stack pointer use, which is described in the section “Handler Mode and Stack Pointers” below.

Instruction Execution versus Debugging

Normally the CPU runs non-stop, executing a program instruction by instruction unless it is in sleep mode or halted. However, there is a debug state in which the CPU does not execute instructions, but is instead controlled by a debugger circuit. Because this debugger hardware has full access to registers and memory, the target program does not need to be modified, simplifying the development tools.

Exceptions and Interrupts

Exceptions and interrupts are critical tools for making an embedded system responsive while supporting concurrent operation of hardware and software. In the Arm programmer’s model, interrupts are a type of exception.

Events such as hardware signals or anomalous program conditions can trigger exceptions. A peripheral or external device sends a hardware signal to the exception controller hardware to indicate that an event has occurred. The processor then services (handles) the exception with these steps:

- Pauses the program.
- Saves context information, such as registers and which instruction to execute next in the current program.
- Determines which handler (also called a service routine) to run. Each type of exception or interrupt can have a separate handler.
- Runs the code for the handler.
- Restores the context information that was saved previously. The processor then resumes executing the current program where it left off.

The CPU performs most of this work automatically in hardware, making the system more responsive. The only work performed in software is running the handler.

Exceptions provide efficient event-based processing, unlike polling that can waste many CPU cycles. Exceptions provide a quick response to events regardless of program complexity, location,

or processor state. They enable many multitasking embedded systems to be responsive without needing to use a task scheduler or kernel.

Most interrupts and exceptions are *asynchronous*, meaning they can occur almost anywhere in the program. Note that parts of the program may disable interrupts temporarily, which will keep the ISR from running until after interrupts are enabled. Although there are methods to trigger an interrupt or an exception in software, we will not cover them here.

asynchronous

Activities which are not synchronized with each other, or a protocol which does not send clocking information

CPU Exception Handling Behavior

Handler Mode and Stack Pointers

The CPU can operate in one of two modes. It normally operates in Thread mode, but switches to Handler mode when performing exception processing. The mode determines which stack pointer (MSP or PSP) is used.

- When the CPU is in the Handler mode, SP refers to the MSP.
- When the CPU is in Thread mode, SP can refer to either MSP or PSP. The special CPU register called CONTROL has a flag field called SPSEL that selects which stack pointer to use. If SPSEL is zero, SP refers to the main stack pointer. This is the value after the CPU is reset. If SPSEL is one, SP refers to the process stack pointer.

In systems with a kernel or operating system, threads will use the PSP and the OS, and exception handlers will use the MSP. In systems without such support, only the MSP is used.

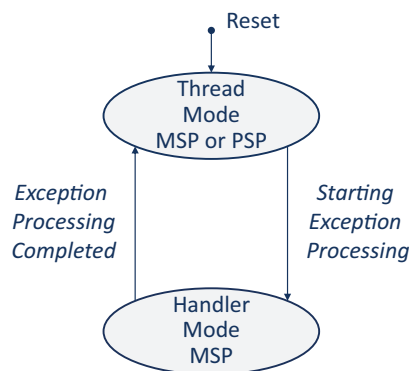


Figure 4.15 CPU operating mode changes when handling exceptions and interrupts.

Entering a Handler

The CPU performs the following steps in hardware in order when an enabled exception is requested. The Cortex-M0 processor takes sixteen cycles (unless memory is slow enough to require wait states).

1. Complete executing the current instruction, except in special cases. Some instructions (**LDM**, **STM**, **PUSH**, **POP**, and **MULS**) are slow and would delay the exception handling significantly. Those instructions are abandoned to allow prompt exception handling, and then restarted after the exception handling completes.
2. Push part of the processor's context onto the current stack (either MSP or PSP), shown in Figure 4.16. Eight 32-bit registers are pushed: the program status register (xPSR), the program counter (the return address), the LR (or R14), R12, R3, R2, R1, and R0. Recall that the stack grows toward smaller addresses. Figure 4.16 shows how critical processor registers that hold execution context for the interrupted code are stored on the stack after performing these steps.
3. Switch the processor to Handler mode and start using MSP.
4. Load the PC with the address of exception handler from the vector table, based on the type of exception. We will examine this shortly.
5. Load LR with **EXC_RETURN** code to select which mode and stack to use after completing the exception processing. The codes are listed in Table 4.2.

	Memory Address	Contents
	0x2000 0ffc	Free space
<i>SP upon entering exception handler</i> →	0x2000 1000	Saved R0
	0x2000 1004	Saved R1
	0x2000 1008	Saved R2
	0x2000 100c	Saved R3
	0x2000 1010	Saved R12
	0x2000 1014	Saved LR
	0x2000 1018	Saved PC
	0x2000 101c	Saved xPSR
<i>SP before entering exception handler</i> →	0x2000 1020	Data

Figure 4.16 Stack changes upon preparing to enter an exception handler.

Table 4.2 Descriptions of Exception Return Codes.

EXC_RETURN Code	Return stack	Description
0xFFFF_FFF1	0 (MSP)	Return to Handler mode with MSP
0xFFFF_FFF9	0 (MSP)	Return to Thread mode with MSP
0xFFFF_FFFD	1 (PSP)	Return to Thread mode with PSP

6. Load the IPSR with the number of the exception being processed. For an **IRQ**, the exception number is 16 + the **IRQ** number.

The CPU can now start executing the code of exception handler.

Exiting a Handler

The CPU performs these steps in order when exiting an exception handler. The first step is a software instruction, while the remainder are hardware operations.

1. Execute an instruction that triggers exception return processing. There is no “return from interrupt” instruction for ARMv6-M processors. Instead, we use an instruction to update the PC with a special exception return code (**EXC_RETURN**) that triggers the CPU’s exception processing hardware. One option is a branch indirect to the link register (**BX LR**). Another option is to pop the exception return code from the stack into the PC.
2. On the basis of the exception return code, the CPU selects either the main or process stack pointer, and also selects either Handler or Thread mode.
3. The CPU restores the context from that stack, as shown in Figure 4.17. This consists of the registers that were saved previously: the xPSR, the program counter (the return address), the LR (or R14), R12, R3, R2, R1, and R0.
4. The CPU has now restored the processor context and will resume execution of code at the address that has been restored to the PC.

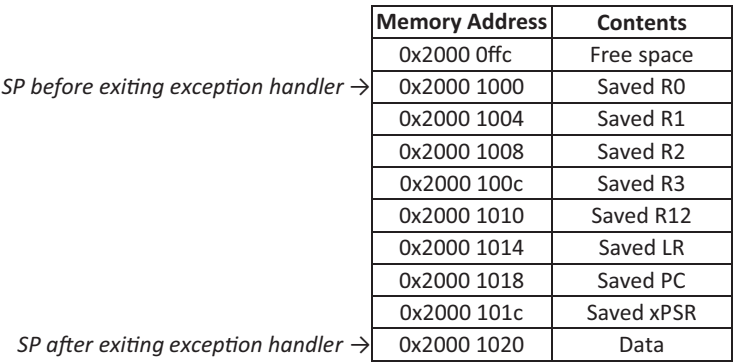


Figure 4.17 Stack changes upon preparing to exit an exception handler.

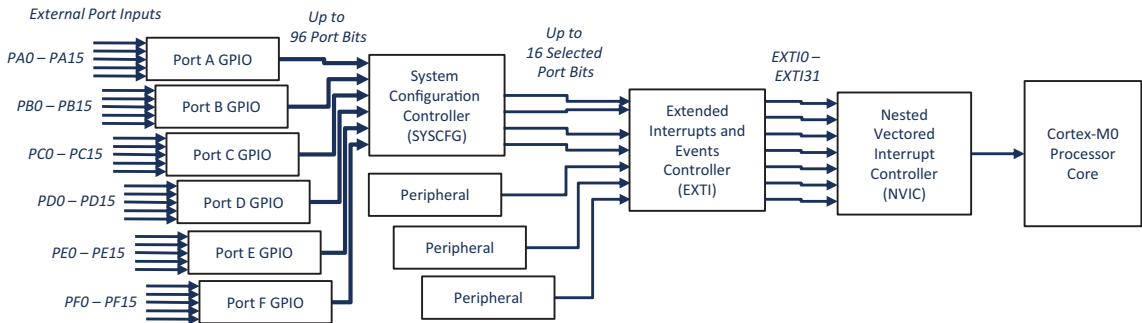


Figure 4.18 Overview of hardware for interrupt system in STM32F091RC MCU.

Hardware for Interrupts and Exceptions

Hardware Overview

Figure 4.18 shows an overview of the hardware involved in recognizing interrupts. At the left are external input signals and peripherals that can generate interrupt requests. These requests flow to the right through the hardware and trigger the CPU core to execute an exception handler routine.

The SYSCFG module selects up to sixteen external input signals and routes them to the EXTI (Extended Interrupts and Events Controller) module. To simplify the interrupt system, it is designed so that only sixteen inputs can request an interrupt: one for each bit (0, 1, 2, to 15). This applies across all six ports; for example, the hardware doesn't allow both PA3 and PE3 to request interrupts.

The EXTI module monitors each of these signals for a valid trigger condition, such as rising edge, falling edge, or either edge. When a trigger occurs on an enabled signal, the EXTI module asserts the corresponding output signal (EXTI0 through EXTI15). The EXTI module also monitors enabled signals from internal MCU peripherals and asserts corresponding output signals (EXTI16 through EXTI30).

The NVIC (Nested Vector Interrupt Controller) can monitor each of these interrupt request signals. If any enabled interrupts are requested, the NVIC selects the one with the highest priority and directs the CPU to start executing its handler. Some exception handlers service multiple types of interrupt requests or exceptions. The NVIC contains control registers to enable and prioritize interrupts.

Finally, on the right is the Cortex-M0 processor core, which can be directed to respond to an IRQ and execute its handler. Within the control register PRIMASK, the PM flag determines whether interrupts (and certain other exceptions) are recognized or ignored. Setting PM to one will cause the CPU to ignore these interrupts. PM is cleared to zero when the processor is reset.

Exception Sources, Vectors, and Handlers

Each possible exception source (or group of sources) has a **vector** to specify its handler routine. The starting addresses for these handlers are stored in a **vector table**. Table 4.3 shows information (including the vector address) for Cortex-M system exceptions, and Table 4.4 shows similar information for microcontroller-specific interrupts. Each vector is four bytes long in order to hold the 32-bit address of the corresponding handler routine. Each vector starts on an address that is a multiple of four, starting with address 0x0000 0004.

vector

Address of an exception handler

vector table

Table of vectors used to process different exceptions

Cortex-M0 Exception Sources

The Cortex-M0 CPU core can generate several types of exceptions, some of which are listed in Table 4.3. These exceptions occur when the CPU starts up, when errors occur, and when system services are requested. Two critical exceptions are Reset, which occurs when the processor first starts running (after being powered up or reset) and HardFault, which occurs when software tries to perform an illegal operation.

Table 4.3 Selected Cortex-M0 Exception Sources and Descriptions

Vector Address	Vector #	Name	Description
0x0000_0004	1	Reset	CPU reset
0x0000_0008	2	NMI	Nonmaskable interrupt
0x0000_000C	3	HardFault	Hard fault error
0x0000_002C	11	SVCall	Call to supervisor with SVC instruction
0x0000_0038	14	PendSV	System-level service request
0x0000_003C	15	SysTick	System timer tick

Table 4.4 STM32F091RC Interrupt Sources and Descriptions

Vector address	Vector #	IRQ	Description
0x0000_0040	16	0	Window Watchdog
0x0000_0044	17	1	PVD and VDD supply comparator
0x0000_0048	18	2	Real Time Clock
0x0000_004C	19	3	Flash
0x0000_0050	20	4	RCC and CRS
0x0000_0054	21	5	EXTI line of pins Px0 and Px1
0x0000_0058	22	6	EXTI line of pins Px2 and Px3
0x0000_005C	23	7	EXTI line of pins Px4 to Px15
0x0000_0060	24	8	Touch Sensing
0x0000_0064, 68, 6C	25–27	9–11	Direct Memory Access
0x0000_0070	28	12	Analog to digital converter and comparators
0x0000_0074-98	29–38	13–22	Timers
0x0000_009C, A0	39–40	23–24	I ² C communication
0x0000_00A4, A8	41–42	25–26	SPI communication
0x0000_00AC, B0, B4	43–45	27–29	USART communication
0x0000_00B8	46	30	CEC and CAN communication
0x0000_00BC	47	31	USB communication

STM32F091RC Interrupt Sources

Many of the STM32F091RC MCU peripherals can request interrupts and are shown in Table 4.4. Vectors for interrupts start with IRQ0 at address 0x0000 0040. Further details can be found in the MCU reference manual [6].

Vector Table Definition and Handler Names

CMSIS-CORE specifies standard names for system exception handlers. Different microcontrollers will have different peripherals, so the interrupt handler's names and vectors are defined in the MCU-specific startup code. The STM32F091RC MCU uses the assembly language file `startup_stm32f091xc.s`, shown in Listing 4.2. The DCD symbol tells the assembler to define a constant data word with the value specified, which in this case is the address of the specified handler.

```
__Vectors
DCD __initial_sp           ; Top of Stack
DCD Reset_Handler         ; Reset Handler
DCD NMI_Handler           ; NMI Handler
DCD HardFault_Handler     ; Hard Fault Handler
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD SVC_Handler           ; SVCcall Handler
DCD 0                     ; Reserved
DCD 0                     ; Reserved
DCD PendSV_Handler        ; PendSV Handler
DCD SysTick_Handler       ; SysTick Handler

    ; External Interrupts
DCD WWDG_IRQHandler        ; Window Watchdog
DCD PVD_VDDIO2_IRQHandler  ; PVD through EXTI Line detect
DCD RTC_IRQHandler         ; RTC through EXTI Line
DCD FLASH_IRQHandler       ; FLASH
DCD RCC_CRs_IRQHandler     ; RCC and CRS
DCD EXTI0_1_IRQHandler     ; EXTI Line 0 and 1
DCD EXTI2_3_IRQHandler     ; EXTI Line 2 and 3
DCD EXTI4_15_IRQHandler    ; EXTI Line 4 to 15
DCD TSC_IRQHandler         ; TS
DCD DMA1_Ch1_IRQHandler    ; DMA1 Ch. 1
DCD DMA1_Ch2_3_DMA2_Ch1_2_IRQHandler ; DMA1 Ch. 2,3 & DMA2 Ch. 1,2
DCD DMA1_Ch4_7_DMA2_Ch3_5_IRQHandler ; DMA1 Ch. 4-7 & DMA2 Ch. 3-5
DCD ADC1_COMP_IRQHandler   ; ADC1, COMP1 and COMP2
DCD TIM1_BRK_UP_TRG_COM_IRQHandler ; TIM1 Events
DCD TIM1_CC_IRQHandler     ; TIM1 Capture Compare
DCD TIM2_IRQHandler        ; TIM2
DCD TIM3_IRQHandler        ; TIM3
DCD TIM6_DAC_IRQHandler    ; TIM6 and DAC
DCD TIM7_IRQHandler        ; TIM7
```

```

DCD TIM14_IRQHandler ; TIM14
DCD TIM15_IRQHandler ; TIM15
DCD TIM16_IRQHandler ; TIM16
DCD TIM17_IRQHandler ; TIM17
DCD I2C1_IRQHandler  ; I2C1
DCD I2C2_IRQHandler  ; I2C2
DCD SPI1_IRQHandler   ; SPI1
DCD SPI2_IRQHandler   ; SPI2
DCD USART1_IRQHandler ; USART1
DCD USART2_IRQHandler ; USART2
DCD USART3_8_IRQHandler ; USART3, USART4, USART5, USART6, USART7, USART8
DCD CEC_CAN_IRQHandler ; CEC and CAN

```

Listing 4.2 Vector table for STM32F091RC MCU in `startup_stm32f091xc.s`.

Input and Peripheral Interrupt Configuration

To use interrupts, we must configure the hardware shown in Figure 4.18. We will examine each component and start on the left. We first examine how to configure a peripheral to generate an interrupt request. Different types of peripherals have interrupt configuration options. Here we will examine the STM32F091RC GPIO module as it is used in our example system. Interrupt configuration for other peripherals is covered in later chapters.

GPIO and SYSCFG

The GPIO modules can provide up to 96 port bit signals to the SYSCFG module. The SYSCFG module selects up to sixteen of these and routes them to the EXTI module, as shown in Figure 4.19. STM32 microcontrollers allow use of any I/O pin as an external interrupt line. The GPIO for that pin must be configured so that its input buffer is enabled, forwarding its signal to the SYSCFG module. The input buffer is enabled by setting the port bit as a digital input or output or alternate

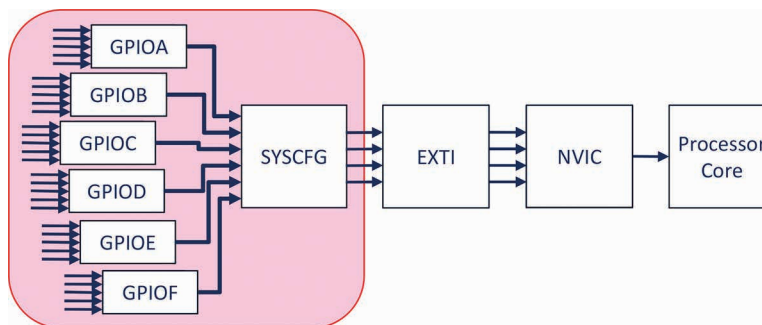


Figure 4.19 Configure GPIO inputs and SYSCFG to generate interrupts.

function, and is disabled for analog or oscillator function. Listing 4.3 shows the code to enable Port C and enable PC7 and PC13 as digital inputs with pull-up resistors.

```
// Enable peripheral clock of GPIOC (for switches)
RCC->AHBENR |= RCC_AHBENR_GPIOCEN;
// Configure PC7 and PC13 in input mode
MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER7, ESF_GPIO_MODER_INPUT);
MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, ESF_GPIO_MODER_INPUT);
// Enable pullup on each input with 01.
MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR7, 1);
MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR13, 1);
```

Listing 4.3 GPIO initialization code sets up PC7 and PC13 as digital inputs with pull-up resistors (step one of four).

The interrupt hardware is designed such that we can use up to 16 pins as external interrupt inputs. Six pins are connected to a multiplexer for each external interrupt signal, as shown in Figure 4.20 for EXTI0 through EXTI4. There is one signal for each bit position in all ports (0 through 15). The EXTI0 signal line can only be connected to one of six possible inputs: PA0,

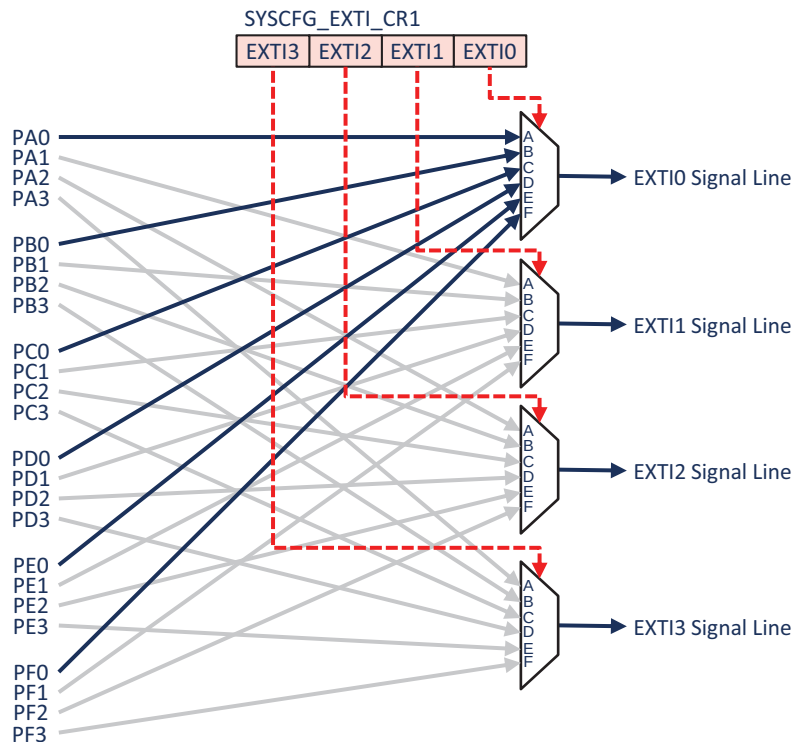


Figure 4.20 Each EXTI signal line can be connected to the corresponding input bit of ports PA through PF, as controlled by fields in register SYSCFG_EXTI_CR1. EXTI0 through EXTI3 are shown here, with inputs of EXTI0 emphasized. EXTI4 through EXTI15 are selected by similar multiplexers, controlled by SYSCFG_EXTI_CR2 through SYSCFG_EXTI_CR4.

PB0, PC0, PD0, PE0 and PF0. This makes it impossible to program PA0 and PB0 to support interrupts at the same time. It is, however, possible to use PA0 and PB1 as interrupt lines simultaneously.

Table 4.5 SYSCFG external interrupt configuration registers.

Register Name	Interrupt Lines	CMSIS-CORE Register Name	Bits 15-12	Bits 11-8	Bits 7-4	Bits 3-0
SYSCFG_EXTICR1	0-3	SYSCFG->EXTICR[0]	EXTI3	EXTI2	EXTI1	EXTI0
SYSCFG_EXTICR2	4-7	SYSCFG->EXTICR[1]	EXTI7	EXTI6	EXTI5	EXTI4
SYSCFG_EXTICR3	8-11	SYSCFG->EXTICR[2]	EXTI11	EXTI10	EXTI9	EXTI8
SYSCFG_EXTICR4	12-15	SYSCFG->EXTICR[3]	EXTI15	EXTI14	EXTI13	EXTI12

Table 4.6 EXTI field configuration values to select input port for interrupt line.

EXTIx	Port Bit Selected
0	PAx
1	PBx
2	PCx
3	PDx
4	PEx
5	PFx
other	reserved

The multiplexer for interrupt line x is controlled by a field $EXTIx$ in one of the four SYSCFG_EXTICR control registers. Note that the CMSIS abstraction layer in `stm32f091xc.h` refers to these four registers as an array, with numbering starting at *zero* (not one). Please refer to Table 4.5 for details and be careful with your code.

Table 4.6 shows the value needed to connect an interrupt line to the pin of the given port.

Listing 4.4 shows code to configure SYSCFG to support switch 1 (PC13) and switch 2 (PC7). First the clock for SYSCFG is enabled by setting the field SYSCFGEN in the RCC_APB2ENR register.

```
// Enable peripheral clock for SYSCFG
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
// Select Port C for bits for SW1 and SW2
// SW1 is at Port C bit 13
MODIFY_FIELD(SYSCFG->EXTICR[3], SYSCFG_EXTICR4_EXTI13, 2);
// SW2 is at Port C bit 7
MODIFY_FIELD(SYSCFG->EXTICR[1], SYSCFG_EXTICR2_EXTI7, 2);
```

Listing 4.4 SYSCFG initialization code sets up PC13 and PC7 as external interrupt lines (step two of four).

Now we need to connect interrupt line 13 to PC13. First we need the name of the register to access. SYSCFG_EXTICR4 controls interrupt line 13 with the EXTI13 field, as seen in Table 4.5. However, in our code we need to use the CMSIS-CORE name of `SYSCFG->EXTICR[3]` (not `SYSCFG->EXTI[4]`).

Next, we need to specify the bit field. The file `stm32f091xc.h` defines symbols for bit fields in registers. In this case we are looking for the bit definition for the EXTI13 field of the SYSCFG_EXTICR4 register, which is `SYSCFG_EXTICR4_EXTI13`.

Finally, we need to specify the value to shift and write to the bit field. Table 4.6 shows the value for PCx is 2. We will use the `MODIFY_FIELD` macro to write a value 2, selecting PC. We follow a similar process to connect switch 2 (PC7) to EXTI7.

EXTI

Next in Figure 4.21, the extended interrupts and events controller (EXTI) determines if an interrupt can be generated and under which conditions. Figure 4.22 shows the hardware logic used to identify valid interrupt conditions from input signals.

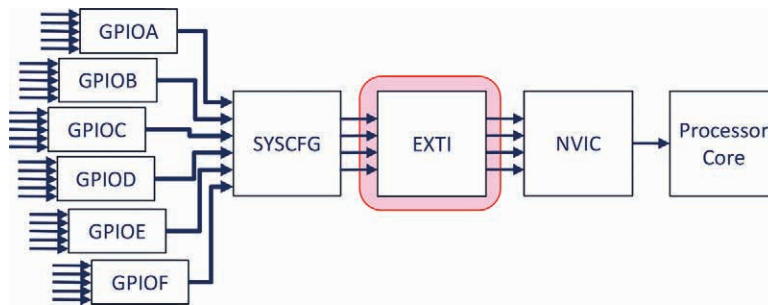


Figure 4.21 Configure EXTI to manage interrupt requests.

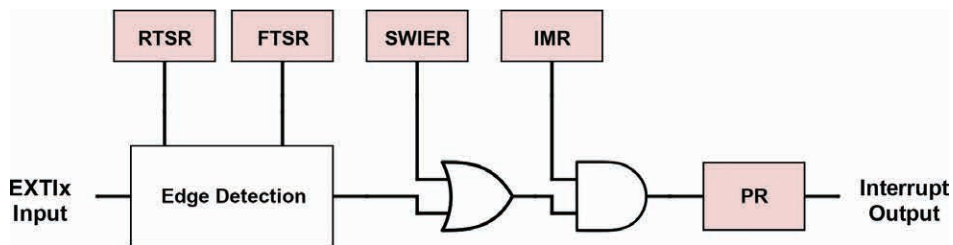


Figure 4.22 EXTI hardware for edge detection, software triggering, masking and pending status.

Figure 4.23 shows a map of the registers EXTI uses to configure and monitor the interrupt lines. Each register has a one-bit field for each interrupt: bit x is associated with EXTI x . The sixteen available external interrupts use bits 0 through 15, while internal interrupts use bits 16 through 31 (see the reference manual’s section “External and internal interrupt/event line mapping”).

	Bit/Interrupt Source																															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI_IMR	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
EXTI_EMR	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
EXTI_RTSR	●	Res.	Res.	Res.	Res.	Res.	Res.	Res.	●	●	●	●	●	Res.	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
EXTI_FTSR	●	Res.	Res.	Res.	Res.	Res.	Res.	Res.	●	●	●	●	●	Res.	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
EXTI_SWIER	●	Res.	Res.	Res.	Res.	Res.	Res.	Res.	●	●	●	●	●	Res.	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
EXTI_PR	●	Res.	Res.	Res.	Res.	Res.	Res.	Res.	●	●	●	●	●	Res.	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●

Figure 4.23 EXTI registers use one bit for each interrupt source.

We start at the input (left side) of the circuit shown in Figure 4.22. The first set of logic determines which conditions might trigger an interrupt. The edge detect circuit determines which types of input signal edges can trigger an interrupt.

- If a bit's Rising Trigger Selection Register (EXTI_RTSR) field is 1 then signal's rising edge (0 to 1) can trigger an interrupt. If 0 it cannot.
- If a bit's Falling Trigger Selection Register (EXTI_FTSR) field is 1 then signal's falling edge (1 to 0) can trigger an interrupt. If 0 it cannot.
- Setting both RTSR and FTSR for a bit will trigger interrupts on both edges.

External interrupts can be emulated by using software to write a 1 to the bit's field in the Software Interrupt Event Register (EXTI_SWIER).

The edge detection and software interrupt circuit outputs are combined with an OR gate before being sent to masking logic. If the bit's Interrupt Mask Register (EXTI_IMR) field is zero the interrupt line is disabled (masked), so no interrupts can be generated. A one enables (unmasks) the interrupt line.

```
// Set mask bits for inputs in EXTI_IMR
EXTI->IMR |= MASK(SW1_POS) | MASK(SW2_POS);
// Trigger on both rising and falling edges in EXTI_RTSR and EXTI_FTSR
EXTI->RTSR |= MASK(SW1_POS) | MASK(SW2_POS);
EXTI->FTSR |= MASK(SW1_POS) | MASK(SW2_POS);
```

Listing 4.5 EXTI configuration code lets PC13 (switch 1) and PC7 (switch 2) generate an interrupt on the rising and falling edges of an input signal (step three of four).

Listing 4.5 shows how to enable interrupts for PC13 and PC7 by setting the mask bits in `EXTI->IMR`. Both rising- and falling-edge interrupts are to be detected, so the corresponding bits are set in `EXTI->RTSR` and `EXTI->FTSR`.

The last part of the logic keeps track of whether an interrupt has been requested but not yet serviced. When an enabled (unmasked) interrupt signal is generated, it will set the bit's field in the Pending Register (EXTI_PR) to 1. This will trigger the CPU to execute the exception handler. Software can also poll EXTI_PR to determine if there are any pending interrupts. It is the handler's responsibility to clear this bit's EXTI_PR field back to 0 to indicate the request has been serviced. This is done by writing a one (not zero) to this bit to clear it to zero. With CMSIS-CORE the register is accessed as **EXTI->PR**.

The EXTI has other capabilities as well. It can generate events for MCU use and wake up based on both internal and external signals; see the Interrupts and Events section of the reference manual.

NVIC Operation and Configuration

Next we examine how to configure the NVIC (Figure 4.22), which selects the highest-priority-enabled interrupt request and directs the CPU to execute its ISR. The NVIC supports up to 32 interrupt sources, called IRQ0 through IRQ31. Each source can be configured. Each interrupt source can be enabled, disabled, and prioritized. Pending interrupt processing status can be read and modified.

EXTI Grouping for Handlers

Table 4.4 shows the interrupts which the MCU can generate. Note that the sixteen different EXTI interrupts are grouped so they are handled by one of three (not sixteen) handlers, as shown in Table 4.7.

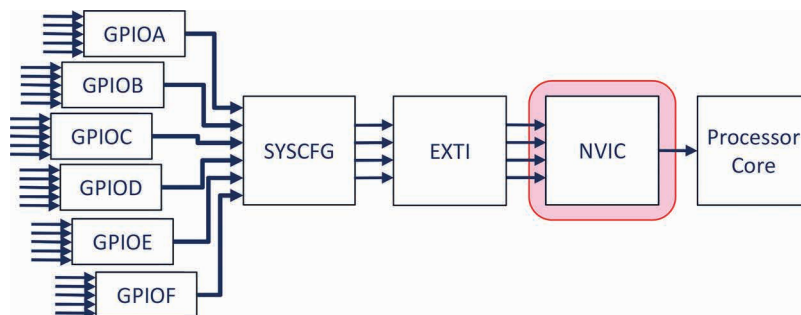


Figure 4.24 Configure NVIC to manage interrupt requests.

Table 4.7 Grouping of external interrupts to exception handlers.

External Interrupts	IRQnum	CMSIS Name for IRQnum	Exception Handler Name
EXTI0 – EXTI11	5	EXTI0_1_IRQn	EXTI0_1_IRQHandler
EXTI12 – EXTI13	6	EXTI12_3_IRQn	EXTI12_3_IRQHandler
EXTI14 – EXTI15	7	EXTI14_15_IRQn	EXTI14_15_IRQHandler

Enable

Each interrupt source can be enabled or disabled separately using the NVIC registers ISER and ICER. Each is 32 bits, with each bit corresponding to an interrupt source. To enable an interrupt source N, write a 1 to bit N in ISER. To disable an interrupt source N, write a 1 to bit N in ICER. Reading ISER or ICER will return the enabled or disabled state for all 32 interrupt sources.

The CMSIS-CORE API provides this interface:

```
void NVIC_EnableIRQ(IRQnum) - Enable interrupts of type IRQnum
void NVIC_DisableIRQ(IRQnum) - Disable interrupt of type IRQnum
```

Values for `IRQnum` shown in Table 4.7 are defined by the CMSIS-compliant device driver, and are listed in the `stm32f091xc.h` file in the section called “Interrupt Number Definitions”. Examples are `TIM2_IRQn`, `USART1_IRQn`, and `EXTI4_15_IRQn`.

Priority

Each exception source has a priority that determines the order in which simultaneous exception requests are handled. The requested exception with the lowest priority number will be handled first. Some exceptions have fixed priorities, such as reset, NMI, and hard fault. Other exceptions (including interrupts) have configurable priorities.

The NVIC contains multiple interrupt priority registers (IPR0 through IPR7). Each IPR has an 8-bit field for each of four interrupt sources. Each field specifies one of four possible priority levels (0, 64, 128, and 192) for that source. Multiple interrupt sources can have the same priority. The CMSIS-CORE API provides the interface here. Note that the `prio` parameter should be set to 0, 1, 2, or 3, as the code shifts the value 6 bits to the left before saving it to the priority field:

```
void NVIC_SetPriority(IRQnum, prio) - Set interrupt source IRQnum to priority
uint32_t NVIC_GetPriority(IRQnum) - Get priority of interrupt source IRQnum
```

Pending

An interrupt is *pending* if it has been requested but has not yet been serviced. The flag is set by hardware when the interrupt is requested. Software can also set the flag to request the interrupt. An interrupt handler that runs must clear its source’s pending IRQ flag or else the handler will run repeatedly.

pending

Requested but not yet serviced (e.g. interrupt)

The CMSIS-CORE API provides this interface:

```
uint32_t NVIC_GetPendingIRQ(IRQnum) - Return 1 if interrupt from IRQnum
void NVIC_SetPendingIRQ(IRQnum) - Set interrupt pending flag for IRQnum
void NVIC_ClearPendingIRQ(IRQnum) - Clear interrupt pending flag for IRQnum
```

```
// Configure enable and mask bits for NVIC IRQ Channel for EXTI
// Interrupt lines 7 and 13 are both serviced by EXTI4_15_IRQ
NVIC_SetPriority(EXTI4_15_IRQn, 3);
NVIC_ClearPendingIRQ(EXTI4_15_IRQn);
NVIC_EnableIRQ(EXTI4_15_IRQn);
```

Listing 4.6 NVIC initialization code configures NVIC to respond to EXTI4_15_IRQ (step four of four).

The code in Listing 4.6 completes the initialization by configuring the NVIC to set the priority level for the specified interrupt, clear any pending interrupts of that type, and finally enable the interrupt.

Processor Exception Masking

Finally, the processor core can be configured to accept or ignore certain types of exceptions, as shown in Figure 4.25. The CPU core's PRIMASK register controls whether interrupts and other exceptions of configurable priority are recognized or not. The PM flag is stored in bit zero of PRIMASK. A PM value of zero allows those exceptions to be recognized by the CPU, whereas a one prevents it. The PM bit is accessed using the **CPSID** and **CPSIE** instructions. Note that when the CPU is reset or powers up the hardware automatically clears PM to zero, so these exceptions are not ignored.

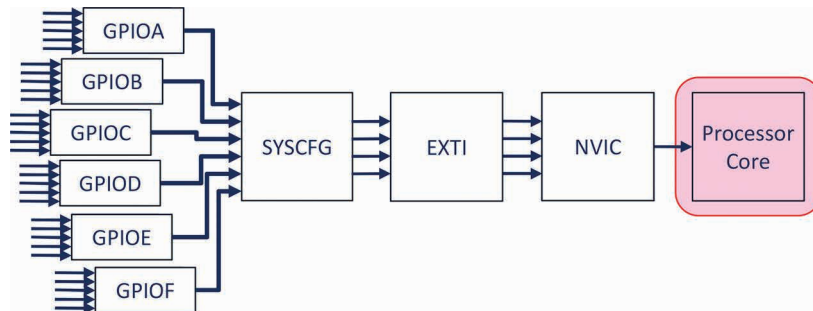


Figure 4.25 Configure processor core to process exceptions.

The CMSIS-CORE API provides these interfaces for accessing the PM flag:

```
void __enable_irq() - Clear PM flag
void __disable_irq() - Set PM flag
uint32_t __get_PRIMASK() - Return value of PRIMASK
void __set_PRIMASK(uint32_t x) - Set PRIMASK to x
```

Safely Masking Interrupts

A function may need to ensure no interrupts are handled during a certain sequence of operations (called a critical section). One approach is to call `__disable_irq()` before the sequence

of operations and `__enable_irq()` after. However, if interrupts were already disabled (for some other reason) before executing the sequence of operations, then it is incorrect to enable them afterward.

The correct approach is shown in Listing 4.7. The code must save the masking state (using `__get_PRIMASK()`) before disabling interrupts (using `__disable_irq()`). After performing the critical section operations, the saved masking state is restored (using `__set_PRIMASK()`).

```
void my_function(void) {
    uint32_t masking_state;
    // Perform non-critical processing
    // ...
    // Get current interrupt masking state
    masking_state = __get_PRIMASK();
    // Disable interrupts
    __disable_irq();
    // Perform critical section operations
    // ...
    // Restore previous interrupt masking state
    __set_PRIMASK(masking_state);
    // Perform more non-critical processing
    // ...
}
```

Listing 4.7 Correct method to prevent interrupts during critical section of code saves and restores interrupt masking state.

Software for Interrupts

When creating software for a system that uses interrupts, we design the system first and then implement it by writing the code. We design the system by identifying which interrupts to use and then deciding how to divide (partition) the work between ISRs and main-line code. We can then write the code to configure hardware to generate and handle interrupts, and finally write the code for the ISRs themselves.

Program Design

Selecting which interrupts to use is generally simple because they are peripheral-specific. Some peripherals have flexible interrupt generation capabilities that can simplify the software design significantly when used appropriately. We will examine these features in later chapters.

Determining how to structure the code in response to the interrupt depends on two major issues: How should we partition work between the ISR and the main-line code? How will the ISR communicate the remaining work to the main-line code? We discuss these next. These topics are covered further in other texts [7] [8] [9].